

SSY191- Project Report

1st Oskar Ulstein
Automation and Mechatronics
Chalmers University of Technology
Gothenburg, Sweden
oskaru@student.chalmers.se

2nd Dharunkumar Senthilkumar
Automation and Mechatronics
Chalmers University of Technology
Gothenburg, Sweden
dhasen@chalmers.se

3rd Soheb Kanon
Automation and Mechatronics
Chalmers University of Technology
Gothenburg, Sweden
soheb@chalmers.se

4th Syed Ahsan Ali Shah
Electrical Engineering
Chalmers University of Technology
Gothenburg, Sweden
syedal@student.chalmers.se

I. INTRODUCTION

The human aspiration to achieve flight has driven centuries of innovation, leading to the development of aerial systems with increasing agility and autonomy. Among these, quadcopters have emerged as highly maneuverable and compact platforms with applications in surveillance, aerial mapping, autonomous delivery, and research.

Despite their potential, quadcopters are underactuated and inherently unstable systems, making their control a challenging task, especially in dynamic and uncertain environments. Addressing these challenges requires precise modeling and robust control strategies.

Model-Based Design (MBD) provides a systematic approach to developing and testing control algorithms by enabling engineers to simulate and analyze system behavior prior to physical implementation. This reduces development time, enhances reliability, and improves system performance.

In this work, a quadcopter is modeled using Simscape and MATLAB/Simulink, capturing the full six degrees of freedom. A complementary filter is implemented to estimate orientation using onboard inertial sensors, and a Linear Quadratic Regulator (LQR) is designed based on a linearized model around hover conditions. The objective is to achieve stable flight and demonstrate real-world control using model-derived strategies.

II. ORIENTATION ESTIMATION

Estimating the orientation of the drone is crucial. The implemented sensors, gyroscope and accelerometer, provide measurements that are susceptible to errors. The gyroscope, which outputs angular velocity, requires numerical integration to estimate angular position. However, due to the discrete nature of sampling, this integration introduces approximation errors that accumulate over time, resulting in drift. Thus, while gyro data is accurate over short periods, it becomes increasingly unreliable over longer durations.

In contrast, the accelerometer can directly provide inclination estimates without the need for integration, using the distribution of acceleration forces relative to gravity. Only the roll (ϕ) and pitch (θ) angles are estimated and can be derived from the accelerometer, as gravity does not influence yaw [1]. These estimates are obtained using trigonometric relationships derived from the orientation of the acceleration vector in the body frame:

$$\theta = \arctan\left(\frac{-a_x}{\sqrt{a_y^2 + a_z^2}}\right), \quad \phi = \arctan\left(\frac{a_y}{a_z}\right) \quad (1)$$

Although accelerometer readings are prone to high-frequency noise, they remain stable over time due to the absence of drift. On the other hand, gyroscope data is smoother in the short term but suffers from cumulative error. To combine the advantages of both sensors, a complementary filter is implemented to merge their data in a balanced way.

A. Implementation of Complementary Filter

The complementary filter utilizes a discretized form of the backward Euler method to fuse gyroscopic and accelerometric estimates. The filter equations for pitch and roll are given by:

$$\hat{\theta}_k = (1 - \gamma) \cdot \theta_{a,k} + \gamma \cdot (\hat{\theta}_{k-1} + h \cdot \omega_{\theta,k}) \quad (2)$$

$$\hat{\phi}_k = (1 - \gamma) \cdot \phi_{a,k} + \gamma \cdot (\hat{\phi}_{k-1} + h \cdot \omega_{\phi,k}) \quad (3)$$

Where:

- h denotes the sampling interval
- $\gamma = \frac{\alpha}{\alpha + h}$ defines the blending factor based on the cut-off frequency α
- $\theta_{a,k}$ and $\phi_{a,k}$ are the pitch and roll angles derived from accelerometer data at time step k
- $\omega_{\theta,k}$ and $\omega_{\phi,k}$ are the angular velocities from the gyroscope for pitch and roll, respectively

This fusion technique effectively reduces noise while preventing drift, yielding reliable orientation estimates throughout the flight.

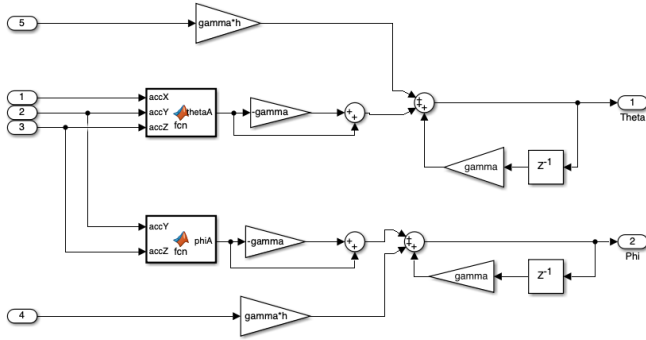


Fig. 1. Simulink model illustrating the implementation of the complementary filter for pitch and roll estimation.

B. Result

The complementary filter was evaluated using data from a pre-recorded test flight. To assess its performance, we compare its output to the result obtained by directly integrating the gyroscope data. This comparison helps determine whether the filter effectively removes drift while preserving the overall signal shape. The filter is tuned using a parameter γ , which ranges from 0 to 1. A value of γ close to 0 places greater weight on the accelerometer data, while a value close to 1 emphasizes the gyroscope data. We found that a $\gamma = 0.99$ effectively removes the drift but also preserves the shape of the signal. See Figure 2 and 3. The yellow graph is the complementary filter while the blue one is the integrated gyro signal.

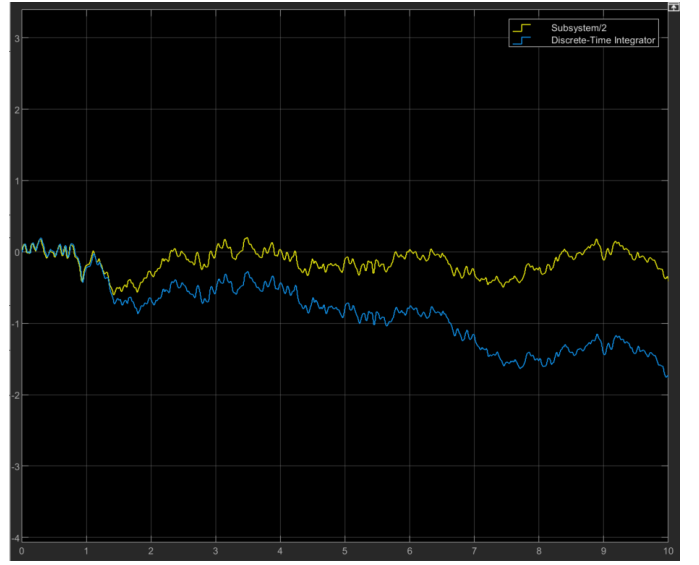


Fig. 2. Estimation of roll angle



Fig. 3. Estimation of pitch angle

III. QUADROTOR PLANT MODELING AND LINEARIZATION

The quadrotor is modeled using Newton–Euler equations for rigid-body dynamics, assuming a rigid body with six degrees of freedom. It follows an X-configuration layout with four rotors positioned symmetrically along two diagonal arms. The system takes motor thrusts as inputs and produces translational and rotational states as outputs. MATLAB was used for system discretization and DLQR controller synthesis.

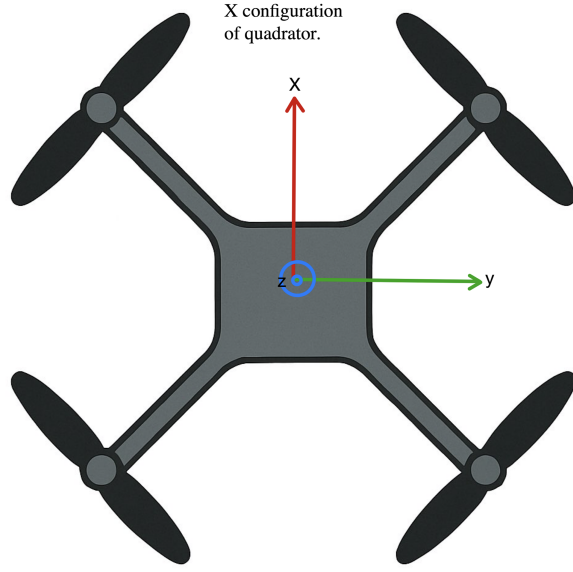


Fig. 4. Quadrotor in X-configuration layout, showing motor numbering and spin direction.

$$\mathbf{u} = [u_1 \quad u_2 \quad u_3 \quad u_4]^T$$

These inputs represent the forces generated by each of the four rotors. The total thrust force applied in the body z-direction is given by:

$$T = \sum_{i=1}^4 u_i$$

This total thrust acts upwards in the body frame and is used to counteract gravity and control vertical motion. The rotor speeds are related to the control inputs by:

$$\omega_i^2 = \frac{u_i}{k_f}$$

Where k_f is the lift coefficient that relates rotor angular speed squared to the produced thrust.

Generated Torques from the Motors: Since the quadrotor is x-configurations the total torque vector generated by the motors about the roll, pitch, and yaw axes is:

$$\boldsymbol{\tau} = \begin{bmatrix} \frac{d}{\sqrt{2}} k_f (-\omega_1^2 - \omega_2^2 + \omega_3^2 + \omega_4^2) \\ \frac{d}{\sqrt{2}} k_f (-\omega_1^2 - \omega_4^2 + \omega_2^2 + \omega_3^2) \\ -k_m (\omega_1^2 + \omega_3^2 - \omega_2^2 - \omega_4^2) \end{bmatrix}$$

This expression describes how the asymmetry in rotor speeds creates torques about each axis. The roll and pitch torques depend on rotor positions and the arm length d , while the yaw torque arises from opposing rotational drag due to rotor spin directions.

Translational Motion Equations: By utilizing the second law of motion the acceleration of the body can be derived by following:

$$\dot{\mathbf{p}} = \mathbf{v}, \quad m \cdot \dot{\mathbf{v}} = -mg \cdot \hat{z} - F\mathbf{v} + {}^B \mathbf{R}_w \begin{bmatrix} 0 \\ 0 \\ -T \end{bmatrix}$$

The first equation states that the position derivative is the velocity. The second is Newton's second law for linear motion, where: $-mg\hat{z}$ is the gravitational force. $-F\mathbf{v}$ is the aerodynamic drag proportional to velocity. ${}^B \mathbf{R}_w \cdot [0 \ 0 \ -T]^T$ transforms the body-frame thrust vector into the world frame.

Rotation Matrix $\mathbf{R}_{wb}$ for Intrinsic Euler Angles: In this model, assuming an intrinsic rotation sequence using Euler angles in the ZYX convention, i.e., yaw (ψ) followed by pitch (θ) and roll (ϕ). The rotation matrix from the body frame to the world frame, denoted $\mathbf{R}_{wb}$, is used to transform vectors between coordinate frames.

$${}^B \mathbf{R}_w = \begin{bmatrix} c_\psi c_\theta & c_\psi s_\theta s_\phi - s_\psi c_\phi & c_\psi s_\theta c_\phi + s_\psi s_\phi \\ s_\psi c_\theta & s_\psi s_\theta s_\phi + c_\psi c_\phi & s_\psi s_\theta c_\phi - c_\psi s_\phi \\ -s_\theta & c_\theta s_\phi & c_\theta c_\phi \end{bmatrix}$$

where $c_\phi = \cos(\phi)$, $s_\phi = \sin(\phi)$, and similarly for θ and ψ . This matrix transforms a vector expressed in the body frame into the world frame.

Rotational Motion Equations:

$$\mathbf{T}(\boldsymbol{\theta}) \cdot \dot{\boldsymbol{\theta}} = \boldsymbol{\omega}$$

This transformation matrix $\mathbf{T}$ relates angular rates in Euler angles to the angular velocity vector in the world frame.

The relationship between the time derivative of Euler angles $\dot{\boldsymbol{\theta}} = [\dot{\phi}, \dot{\theta}, \dot{\psi}]^T$ and the world angular velocity vector $\boldsymbol{\omega} = [\omega_x, \omega_y, \omega_z]^T$ is given by the transformation matrix $\mathbf{T}(\boldsymbol{\theta})$:

$$\boldsymbol{\omega} = \mathbf{T}(\boldsymbol{\theta}) \cdot \dot{\boldsymbol{\theta}}$$

$$\mathbf{T}(\boldsymbol{\theta}) = \begin{bmatrix} 1 & 0 & -\sin(\theta) \\ 0 & \cos(\phi) & \cos(\theta) \sin(\phi) \\ 0 & -\sin(\phi) & \cos(\theta) \cos(\phi) \end{bmatrix}$$

This mapping is needed because the Euler angles are not directly the components of the angular velocity vector. $\mathbf{T}$ encodes the geometric relationship between the rates of change in the Euler angles and the actual angular velocity vector in the body frame.

$$\mathbf{J} \cdot \dot{\boldsymbol{\omega}} = -\boldsymbol{\omega} \times (\mathbf{J}\boldsymbol{\omega}) + \boldsymbol{\tau}$$

This equation models rotational dynamics. The left-hand side is the angular acceleration scaled by the inertia matrix $\mathbf{J}$. The first term on the right is the gyroscopic effect due to existing angular velocity, and the second is the external torque generated by the motors.

A. Linearized Model

To design linear controllers such as DLQR, the nonlinear model must be approximated by a linearize system. This is achieved through Jacobian-based linearization around a hover operating point — in this case, the quadrotor hovering at the origin with zero angular velocity and level orientation:

$$\phi = \theta = \psi = 0, \quad \omega_x = \omega_y = \omega_z = 0, \quad v = 0, \quad T = mg$$

The nonlinear equations are linearized using a first-order Taylor expansion around this equilibrium point. Specifically, we define a state vector:

$$\mathbf{x} = [\phi \quad \theta \quad \omega_x \quad \omega_y \quad \omega_z]^T$$

and input vector:

$$\mathbf{u} = [u_1 \quad u_2 \quad u_3 \quad u_4]^T$$

The system dynamics are approximated as:

$$\dot{\mathbf{x}} \approx \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$$

where $\mathbf{A}$ and $\mathbf{B}$ are Jacobian matrices computed as:

$$\mathbf{A} = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\mathbf{x}_0, \mathbf{u}_0}, \quad \mathbf{B} = \left. \frac{\partial f}{\partial \mathbf{u}} \right|_{\mathbf{x}_0, \mathbf{u}_0}$$

Here, $f(\mathbf{x}, \mathbf{u})$ represents the nonlinear system dynamics. The symbolic differentiation was performed in MATLAB using either analytical expressions or symbolic toolboxes to compute these matrices.

$$\mathbf{x} = [\phi \quad \theta \quad \omega_x \quad \omega_y \quad \omega_z]^T, \quad \mathbf{u} = [u_1 \quad u_2 \quad u_3 \quad u_4]^T$$

State Matrix ($\mathbf{A}$):

$$\mathbf{A} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad (4)$$

The $\mathbf{A}$ matrix represents the linearized dynamics of the system near the hover state. The first two rows relate angular rates to angle derivatives, and the last three describe how angular velocities evolve under torque and inertia coupling.

Input Matrix ($\mathbf{B}$):

$$\mathbf{B} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ -\frac{\sqrt{2}d}{2J_x} & -\frac{\sqrt{2}d}{2J_x} & \frac{\sqrt{2}d}{2J_x} & \frac{\sqrt{2}d}{2J_x} \\ -\frac{\sqrt{2}d}{2J_y} & \frac{\sqrt{2}d}{2J_y} & \frac{\sqrt{2}d}{2J_y} & -\frac{\sqrt{2}d}{2J_y} \\ -\frac{k_m}{J_z k_f} & \frac{k_m}{J_z k_f} & -\frac{k_m}{J_z k_f} & \frac{k_m}{J_z k_f} \end{bmatrix} \quad (5)$$

The $\mathbf{B}$ matrix maps motor inputs to angular accelerations. Each row corresponds to how the inputs affect roll, pitch, and yaw dynamics. The structure reflects the symmetry and rotor layout of the quadrotor.

B. Physical Parameters

Parameter	Value
Mass m	0.027 kg
Gravity g	9.81 m/s ²
Arm length d	0.046 m
Lift constant k_f	1.9796×10^{-9} kg·m/(rad/s) ²
Drag constant k_m	2.5383×10^{-11} kg·m ² /(rad/s) ²
Air drag coefficient F	0.25 N·s/m
Moment of inertia J_x	1.1463×10^{-5} kg·m ²
Moment of inertia J_y	1.6993×10^{-5} kg·m ²
Moment of inertia J_z	2.9944×10^{-5} kg·m ²

TABLE I

PHYSICAL PARAMETERS OF THE QUADROTOR

These physical parameters, summarized in Table I, define the properties of the quadrotor used in this study. They include mass, gravitational acceleration, arm length, lift and drag coefficients, and moments of inertia along each axis. All simulation results presented in the following sections are based on this parameter set.

IV. RESULTS: VERIFICATION OF THE PLANT MODEL

To verify the accuracy of the nonlinear quadrotor model, a simulation was performed where the motor signals are activated at $t = 2$ seconds. The system is initially at rest, and all motors are assumed to start simultaneously, generating symmetric thrust to lift the quadrotor vertically.

At $t = 6$ seconds, motors 2 and 4 are intentionally shut down to observe the resulting dynamics and assess how well the model captures the expected behavior under actuator failure or disturbance.

A. Motor Inputs and Resulting Motion

Figure 5 shows the input signals to all four motors. As seen, all motors generate equal thrust after $t = 2$ seconds, maintaining stable ascent. However, at $t = 6$ seconds, motors 2 and 4 stop producing thrust.

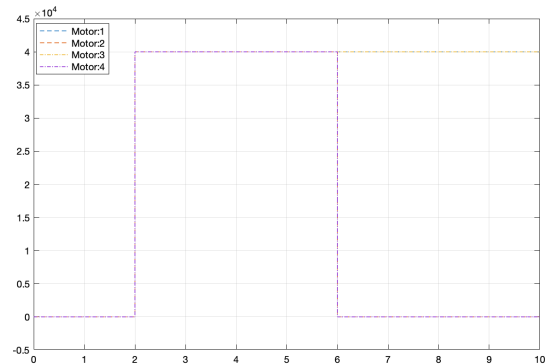


Fig. 5. Motor input signals during simulation. Motors 2 and 4 are deactivated at $t = 4$ seconds.

B. Acceleration Response

Initial the motor is shutdown, the thrust compensates gravity. At $t = 2$ when the motors get signal, resulting acceleration

in z directions are shown in Figure 6 as expected. At $t = 6$ seconds, a significant drop in thrust causes the quadrotor to lose lift and start to rotate due to unbalanced forces.

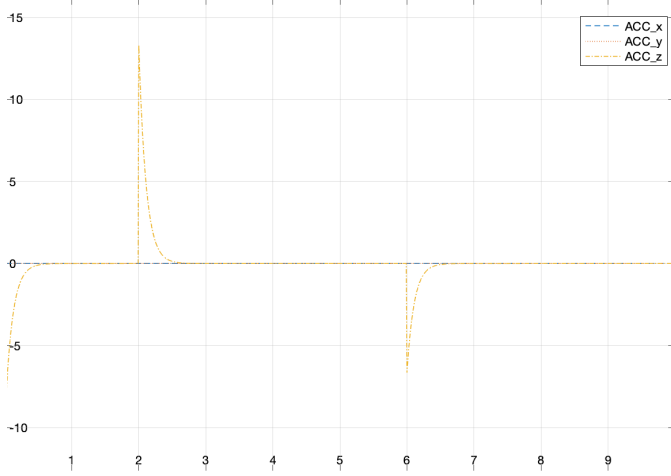


Fig. 6. Linear accelerations in x , y , and z directions.

C. Attitude and Angular Velocity Response

Figure 7 presents the roll, pitch, and yaw angles. The angles remain near zero before the failure, indicating level flight. Following the loss of motors 2 and 4, the quadrotor yaw rapidly due to torque imbalance, validating the model's rotational dynamics.

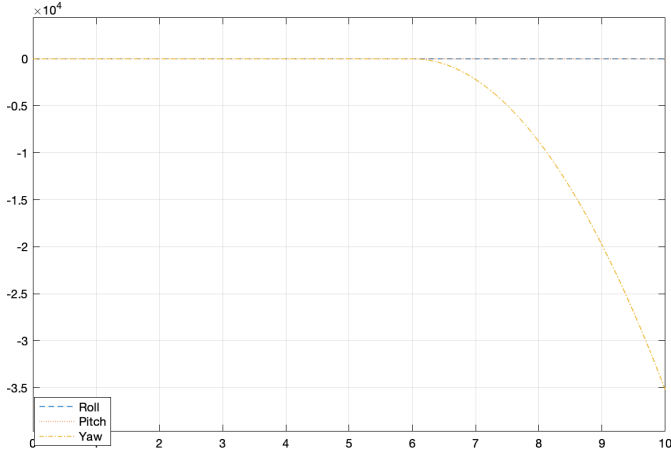


Fig. 7. Roll, pitch, and yaw angles over time.

Figure 8 further confirms this behavior by showing the angular velocities. The velocity in z -directions, which sharply increase $t = 2$ and reach the "steady state" and at $t = 6$ due to the sudden loss of torque equilibrium the velocity in z -directions decreases.

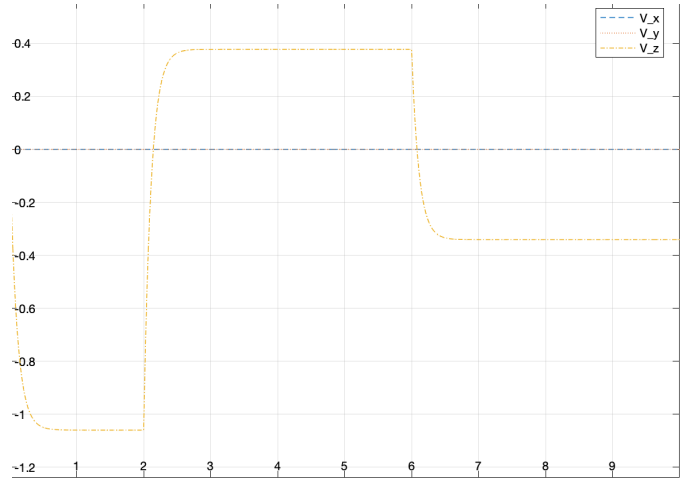


Fig. 8. Angular velocity components ω_x , ω_y , and ω_z .

D. Discussion

These simulation results demonstrate that the plant model correctly captures the dynamics of the quadrotor both in normal flight and under partial motor failure. The loss in thrust leads to realistic changes in acceleration, orientation, and angular velocity. This validates the physical and mathematical modeling presented in the previous section.

V. DESIGN OF LINEAR QUADRATIC CONTROLLER

A. Problem formulation

The objective is to stabilize the roll (ϕ) and pitch (θ) angles and the yaw rate (ω_z) of the system. This is done with a Linear Quadratic Regulator (LQR) controller.

Yaw angle is excluded because it is not reliably controllable using only IMU measurements, as the yaw dynamics are weakly observable and influenced by external disturbances and sensor drift.

The linearized state-space model is expressed as:

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$$

where $\mathbf{x}$ is the state vector and $\mathbf{u}$ is the input vector.

The state vector is chosen as:

$$\mathbf{x} = [\phi \quad \theta \quad \omega_x \quad \omega_y \quad \omega_z]^T$$

where ϕ and θ are roll and pitch angles, and $\omega_x, \omega_y, \omega_z$ are the angular velocities about the x, y, and z axes. Yaw is neglected to reduce model complexity and because the main stabilization goal focuses on roll and pitch dynamics.

The control input vector is:

$$\mathbf{u} = [T_1 \quad T_2 \quad T_3 \quad T_4]^T$$

representing the thrust for each motor.

B. Controllability Analysis

Before designing the LQR controller, it is important to verify that the linearized system is controllable. Controllability ensures that the system states can be driven to the desired equilibrium using the control inputs [2].

The controllability matrix $\mathcal{C}$ is defined as:

$$\mathcal{C} = [\mathbf{B} \quad \mathbf{A}\mathbf{B} \quad \mathbf{A}^2\mathbf{B} \quad \dots \quad \mathbf{A}^{n-1}\mathbf{B}]$$

where n is the number of states.

For the system to be controllable, the rank of $\mathcal{C}$ must be equal to n :

$$\text{rank}(\mathcal{C}) = n$$

Given the state-space matrices $\mathbf{A} \in \mathbb{R}^{5 \times 5}$ and $\mathbf{B} \in \mathbb{R}^{5 \times 4}$, the controllability matrix was computed in MATLAB and found to have full rank:

$$\text{rank}(\mathcal{C}) = 5$$

This confirms that the system is controllable and suitable for the LQR controller design.

C. Design of Cost function and LQR Gain

The LQR controller minimizes the infinite-horizon quadratic cost function:

$$J = \sum_{k=0}^{\infty} (x_k^T \mathbf{Q} x_k + u_k^T \mathbf{R} u_k + 2x_k^T \mathbf{N} u_k) \quad (6)$$

where matrix $\mathbf{Q}$ is a state cost, $\mathbf{R}$ is a control cost matrix, and $\mathbf{N}$ is the cross-term cost matrix. The weighting matrices $\mathbf{Q}$ and $\mathbf{R}$ are chosen to reflect the relative importance of states and control inputs:

- Larger weights on the angles ϕ and θ penalize deviations in roll and pitch more heavily than angular rates ω_x, ω_y and ω_z .
- The matrix $\mathbf{R}$ penalizes large control inputs to avoid excessive actuator effort.

This trade-off helps balance performance and energy efficiency.

The optimal state feedback gain $\mathbf{K}$ is obtained by solving the discrete-time Algebraic Riccati Equation (DARE):

$$\mathbf{P} = \mathbf{A}^T \mathbf{P} \mathbf{A} - (\mathbf{A}^T \mathbf{P} \mathbf{B})(\mathbf{R} + \mathbf{B}^T \mathbf{P} \mathbf{B})^{-1} (\mathbf{B}^T \mathbf{P} \mathbf{A}) + \mathbf{Q}$$

where $\mathbf{P}$ is the positive definite solution.

The control law is:

$$\mathbf{u} = -\mathbf{K}\mathbf{x}$$

with

$$\mathbf{K} = \mathbf{R}^{-1} \mathbf{B}^T \mathbf{P}$$

The gain matrix $\mathbf{K}$, computed using (4), (5), and (7), is obtained in MATLAB via `dlqr()`, with its values dependent on the selected $\mathbf{Q}$ and $\mathbf{R}$.

where the specific numerical values depend on the chosen $\mathbf{Q}$ and $\mathbf{R}$ in (7).

D. Closed Loop system

The closed-loop dynamics under LQR control is:

$$\dot{\mathbf{x}} = (\mathbf{A} - \mathbf{B}\mathbf{K})\mathbf{x}$$

Analyzing the eigenvalues of $\mathbf{A} - \mathbf{B}\mathbf{K}$ on Figure 9 verifies closed-loop stability if all eigenvalues lie within the unit circle in the complex plane. and as it shown all the eigenvalues inside the unit circle

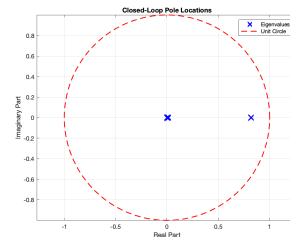


Fig. 9. eigenvalues of closed loop

VI. EVALUATION OF CONTROL DESIGN

Several values for the weighting matrix Q were tested and compared to achieve a good control response. The goal with the tuning were to obtain a fast system response while maintaining minimal overshoot. After evaluating different configurations, the final tuning parameters were selected as:

$$Q = \begin{bmatrix} 400 & 0 & 0 & 0 & 0 \\ 0 & 400 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \quad R = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (7)$$

The simulated response is shown in Figure 10.

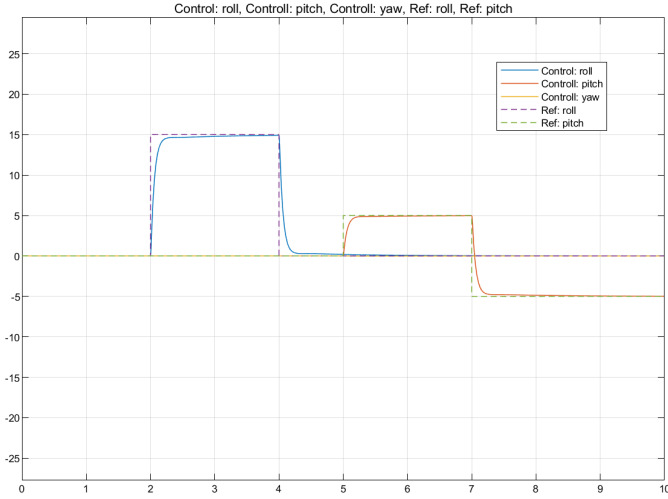


Fig. 10. Simulation with tuned Q and R

However, after testing these parameters on the real system, it was discovered to react too fast, causing instability. This was compensated for by increasing the cost of the velocity states, forcing the system to react more slowly. The final tuning for Q was:

$$Q = \begin{bmatrix} 800 & 0 & 0 & 0 & 0 \\ 0 & 800 & 0 & 0 & 0 \\ 0 & 0 & 100 & 0 & 0 \\ 0 & 0 & 0 & 100 & 0 \\ 0 & 0 & 0 & 0 & 100 \end{bmatrix} \quad (8)$$

With this tuning, the drone can fly, and it is able to compensate for small disturbances. Figure 11 shows a test on the real drone where a disturbance was introduced. The drone efficiently compensated and returned to its steady state. However, for bigger disturbances, the drone is not able to recover and becomes unstable.

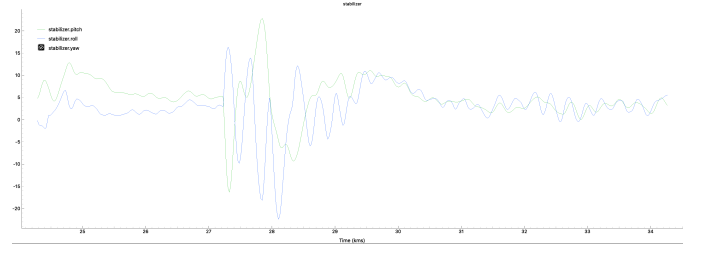


Fig. 11. Pitch and roll data from a test-flight with introduced disturbance.

VII. EMBEDDED IMPLEMENTATION OF THE CONTROLLER

When implementing the controller in C, both the filter and the controller share the same memory, making it essential to prevent race conditions. This requires protecting all shared resources using synchronization mechanisms, such as semaphores. Although this increases implementation complexity, it ensures safe and deterministic behavior when running on a real-time operating system (RTOS).

It is also important to minimize jitter. Jitter refers to variations in task latency between iterations. In control systems, this is particularly problematic, as it undermines the ability to evaluate and ensure the stability of the controller. In this project, jitter was mitigated by using `vTaskDelayUntil()`, which ensures that each task runs at consistent, periodic intervals.

As shown in Figure 12, the filter closely follows the reference model. However, the controller exhibits some deviation from the reference. This deviation may be attributed to estimation delays, mismatched task timing, motor saturation, or the physical limitations of the simulated system.

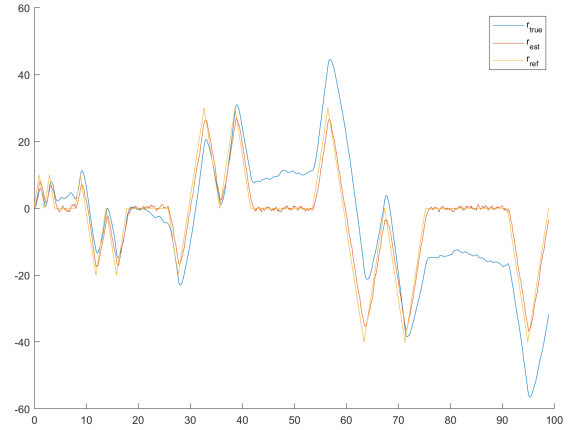


Fig. 12. Simulation with the embedded controller.

VIII. CONCLUSION

In this project, a model-based control system was designed and tested for a small quadrotor. The work includes several key stages: modeling the plant in Simscape, linearizing the system, designing and tuning a controller, estimating states using onboard sensors, and finally validating the results through both simulation and real-world testing.

The focus was on stabilizing the roll and pitch angles, which are critical for maintaining flight balance. Estimation of these angles was achieved using a complementary filter, which fuses data from the gyroscope and accelerometer to provide reliable angle estimates. These estimates were then used as inputs to the controller.

Initially the system model is nonlinear, it was first linearized around an operating point to simplify the control design process. A Linear Quadratic Regulator (LQR) was chosen as the controller due to its ability to balance performance and control effort. The controller was tuned to provide fast response.

Extensive simulations were conducted in Simulink to evaluate the controller under various conditions, including external disturbances, and different thrust levels. Results were sufficient for a stable hover, but did not work great. Large disturbances caused the system to go unstable. For increased performance, more complex control systems are needed. For example, the controller does not include integral action, meaning that it can not account for constant disturbances. The results showed that the LQR controller could stabilize the quadrotor and doesn't achieve the entire desired performance.

One of the most important lessons learned during the project was the gap between simulation and real-world performance. While the controller performed quite well in simulation, the initial real-world tests revealed issues such as instability and fast reactions. These were mitigated by adjusting the cost function to penalize angular velocity more strongly, resulting in a slower but more stable response.

The embedded implementation required careful consideration of real-time constraints, such as avoiding race conditions and minimizing jitter. The embedded implementation of the LQR controller did not work as expected, even in simulation, likely due to timing mismatches, unhandled delays, or incorrect task scheduling in the RTOS environment.

Overall, the project demonstrated that a linear control strategy like LQR can be effectively applied to stabilize a quadrotor, provided the system is properly modeled and tuned.

IX. REFERENCE

[1] G. J. Aparna, C. Kamal, and R. N. Motta, "IMU-based attitude estimation using an adaptive complementary filter,"

in Proc. 2021 Int. Conf. Communication, Information and Computing Technology (ICCICT), Sept. 2021, pp. 1–5, doi: 10.1109/ICCICT50803.2021.9510153.

[2] T. Glad and L. Ljung, *Control Theory: Multivariable and Nonlinear Methods*. Boca Raton, FL, USA: CRC Press, 2014.